

Composer: Dependency Manager

Vitor Mattos

Fotografem, comentem, twitterem!

@VitorMattosRj



Olá! Estava no Rio Agile?

Sim, foi show! Aliá, achei a sua apresentação a melhor. Parabéns!

Não conhecia a ferramenta até então, eu estava iniciando ainda meus estudos sobre BDD. Achei a palestra muito boa mesmo meu nível no assunto sendo muito introdutório.

que trampo foda mano, mt obrg por compartilhar esse conhecimento bro! ou mostrar isso pros devs, talvez isso ajude a engajar os caras a mudar.

Hoje tive a oportunidade de ouvir este grande cara [@VitorMattos](#)
Deixo aqui os meus parabéns, foi uma grande palestra, valeu cada minuto e espero ver mais o seu trabalho fera, parabéns !

22:46

Gostei muito da sua palestra, eu que sou iniciante não entendia bem a importância do composer, ficou muito claro que ele é muito útil e como implementar, adorei.

Obrigado por compartilhar esse tão precioso conhecimento conosco.

Nota 10!!! 🍌🍌🍌🍌🍌

12:49

Quem sou eu?

Realizador de sonhos desde 2003

Amante de opensource

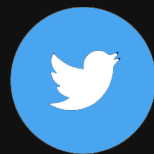
Palestrante

PHP Zend Certified Engineer ([ZEND024235](#))

PHPRio (<https://telegram.me/phprio>)

CTO Lyseon Tech

Redes sociais: (VitorMattos ou VitorMattosRJ)





LYSEONTECH

Falaremos sobre...

No 1. Molto vivace. CHOR. a. d. Op: der Freischütze. 9

Introduzione.

Soprano:
Alto:
Tenori:
Bassi:

Vic - toria, Vic - toria der Meister soll leben, der wacker den Sternlein den Rest hat ge - gehen, der

Só que não



Roteiro

- Introdução e breve histórico
- Uso do packagist como consumidor
- Criação de um pacote (tornando-se um mantenedor)
- Formas de distribuir este pacote
- Criação de scripts

Resumindo:
<https://getcomposer.org>

O que é Composer e para que serve?

Gestão de dependências em projetos PHP inspirado no NPM (node JS) e bundler (Ruby)

Isola dependências externas evitando necessidade de versionamento das mesmas (vendor)

O que é Composer e para que serve?

Gestão de dependências (node JS) e bundler (R) PHP inspirado no NPM

Isola dependências e versionamento das dependências necessidade de



O que são dependências?

Todo artefato de software seu ou de terceiros cuja finalidade é o reaproveitamento de código.

Voltando: O que é Composer e para que serve?

Declaração de quais são todas as dependências externas e suas versões (composer.json)

Voltando: O que é Composer e para que serve?

Isola dependências externas evitando necessidade de versionamento das mesmas



Voltando: O que é Composer e para que serve?

Possui um agregador de repositórios central

Packagist

BrowseSubmitvitormattos



Packagist is the main [Composer](#) repository. It aggregates public PHP packages installable with Composer.

What is Packagist?

Packagist is the default Composer package repository. It lets you find packages and lets Composer know where to get the code from. You can use Composer to manage your project or libraries' dependencies - read more about it on the [Composer website](#).

Voltando: O que é Composer e para que serve?

Possibilidade de uso de repositório privado



Voltando: O que é Composer e para que serve?

Responsabilidade única das partes do projeto

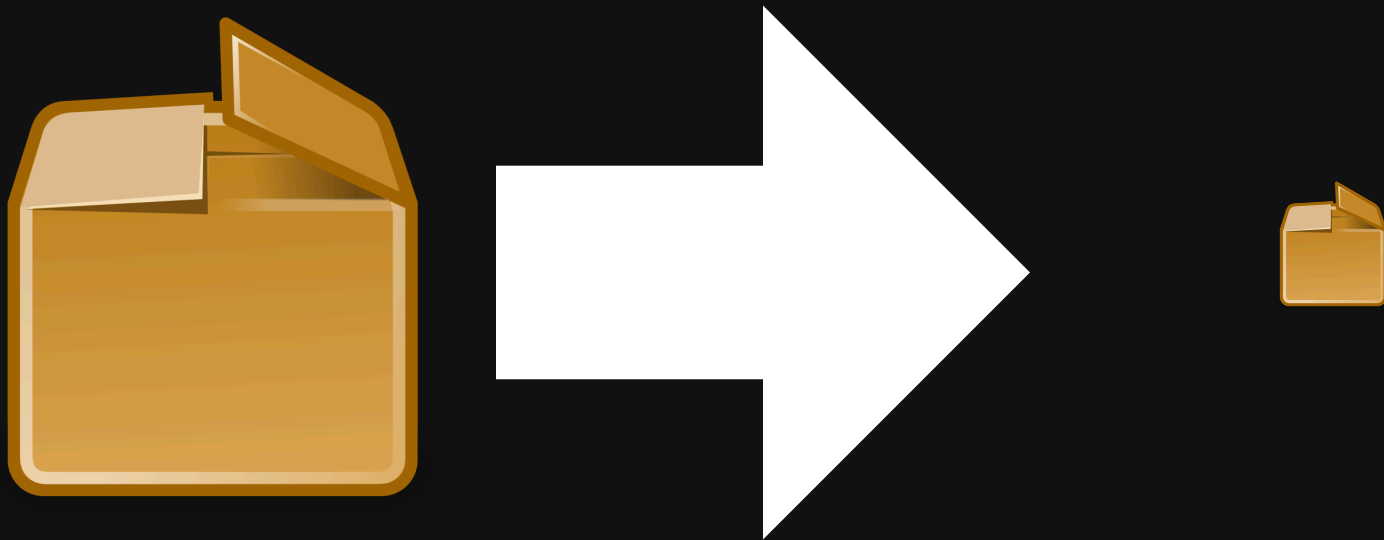


O que é Composer e para que serve?

Você declara somente o que VOCÊ precisa. O composer encontra as bibliotecas declaradas e as instala com todas as suas dependências satisfeitas.

O que é Composer e para que serve?

Reduz o tamanho dos projetos



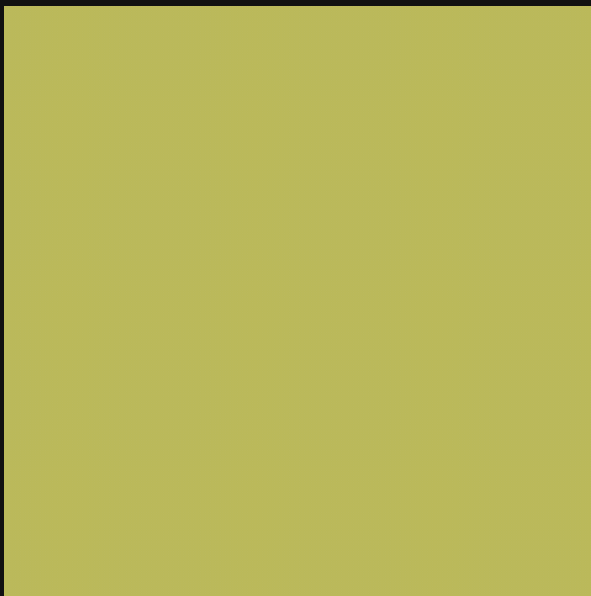
O que é Compose e para que serve?

Reduz esforço



O que é Composer e para que serve?

Facilita redistribuição de projetos



Um pouco de história

Suporte a namespaces no PHP lançado no PHP 5.3 em 2009

```
<?php  
    namespace TheProject;
```

Um pouco de história

Surgimento do PHP-FIG em 2009



PHP Framework Interop Group - PHP-FIG

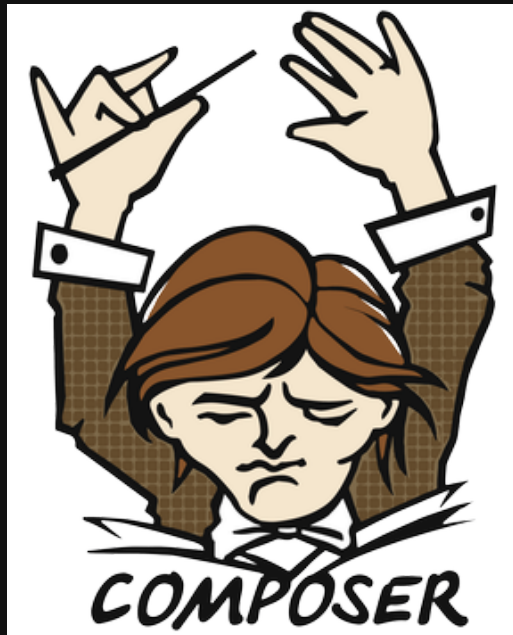
Um pouco de história

Lançamento da PSR-0 com primeiras definições de autoload
em 2010

PSR-0: Autoloading Standard

Um pouco de história

Criação do Composer em 2012 após o PHP-FIG



Um pouco de história

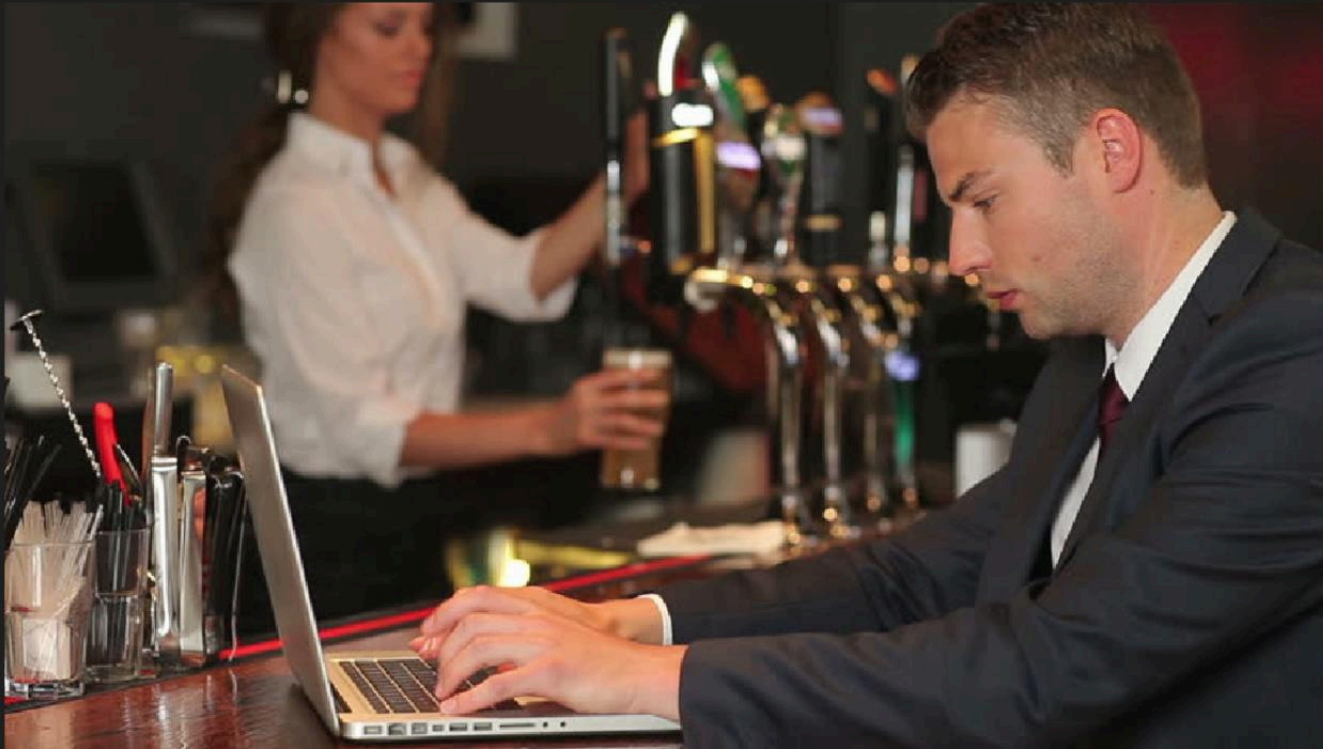
Adoção do Composer por parte de grandes frameworks e ferramentas

Um pouco de história

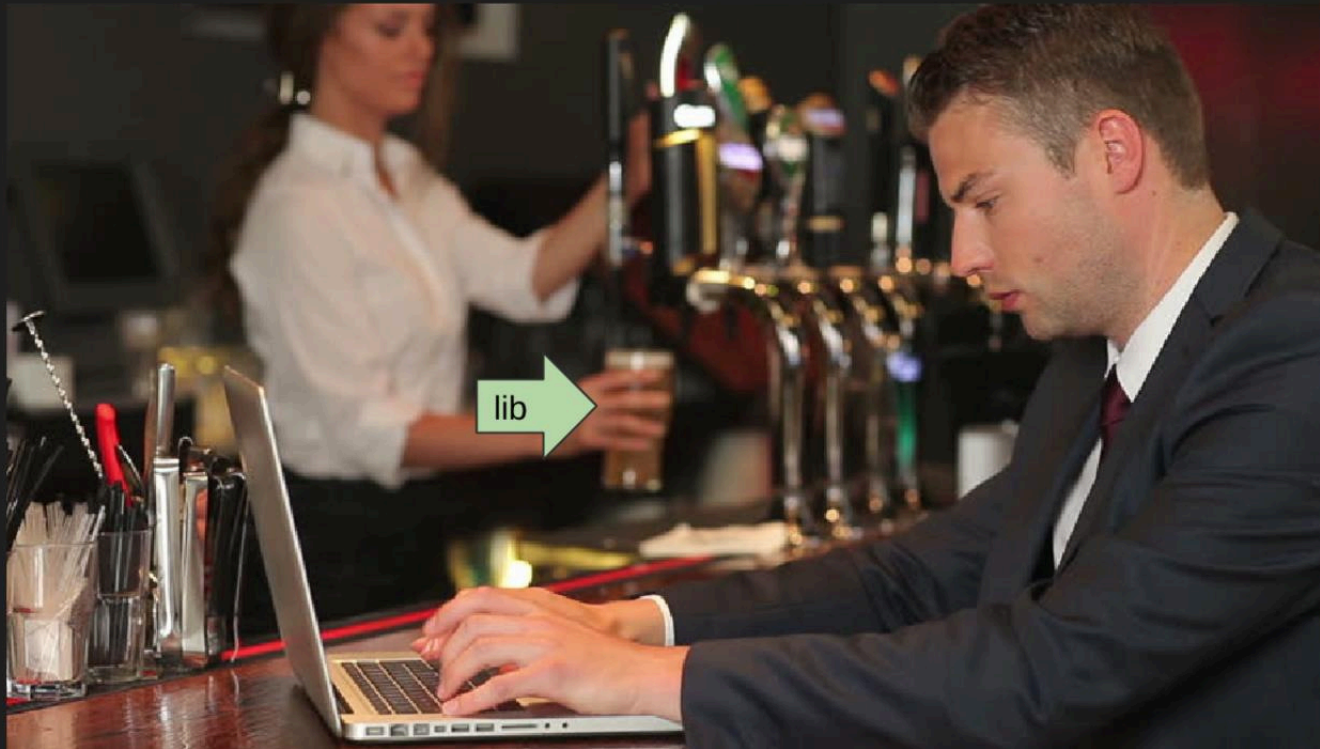
Lançamento da PSR-4 com melhorias nas definições de autoloader
em 2014

PSR-4: Autoloader

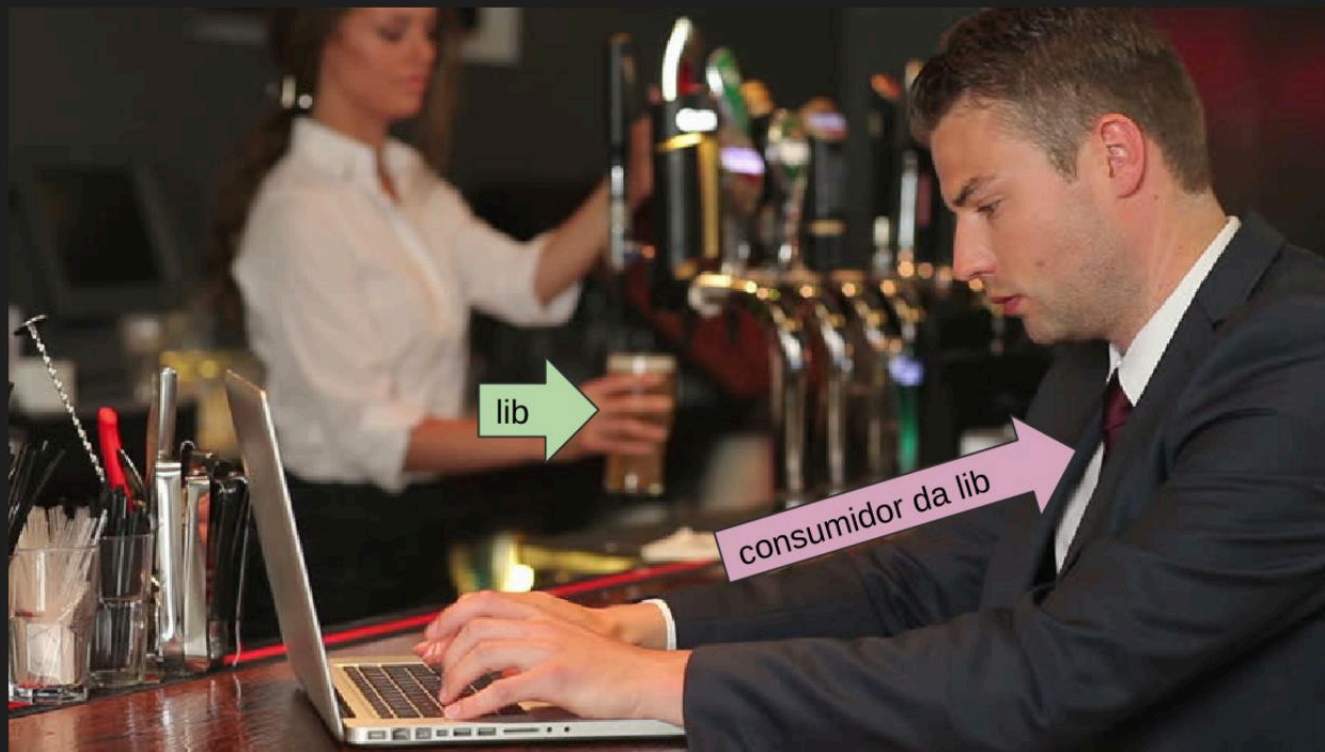
Partes envolvidas



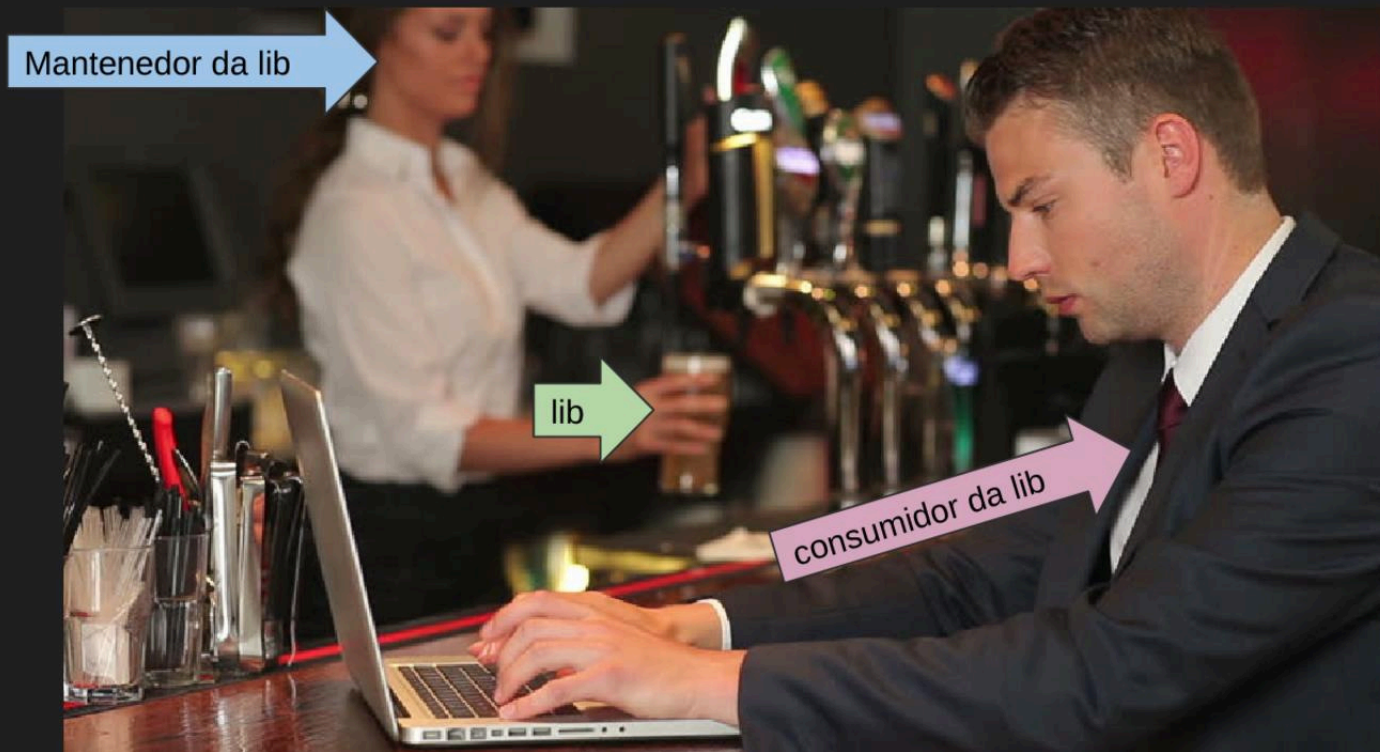
Partes envolvidas



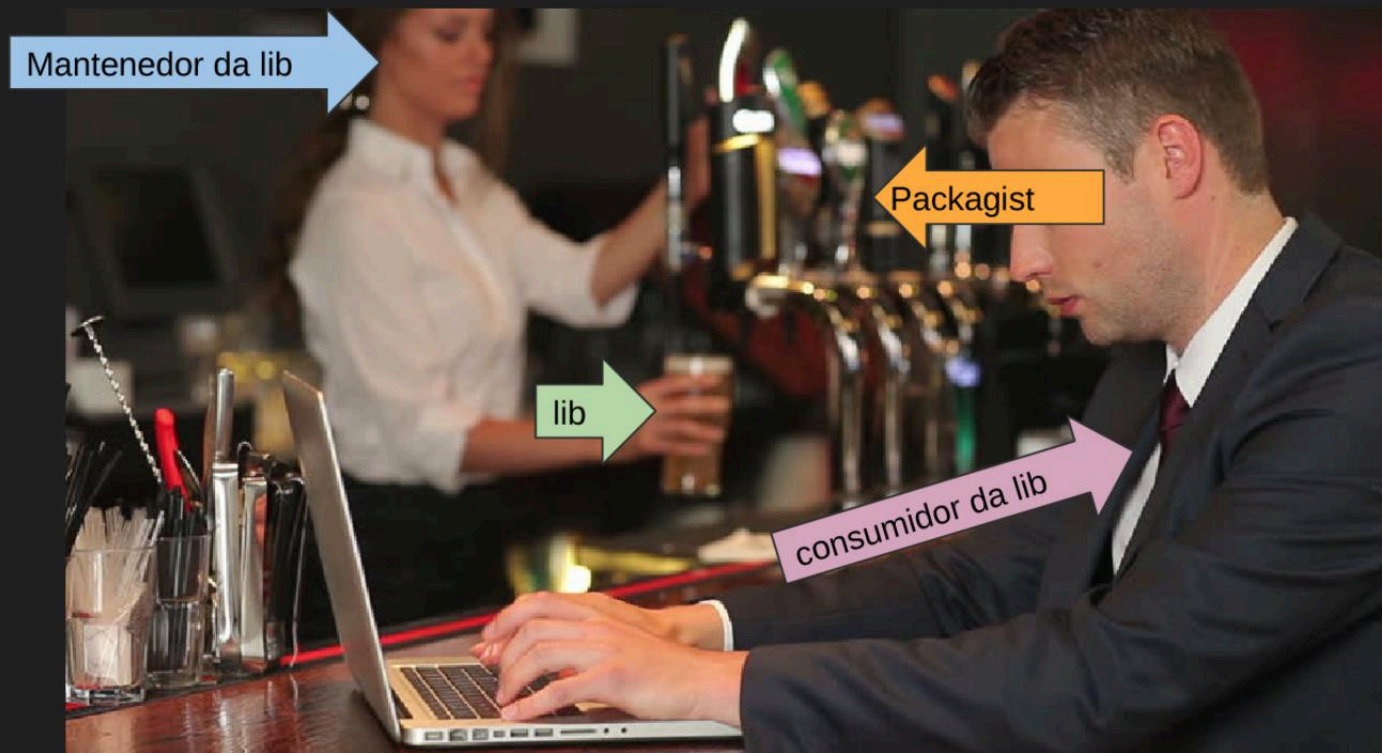
Partes envolvidas



Partes envolvidas



Partes envolvidas



Visão do consumidor: instalação

- É feito em PHP
- Exige no mínimo PHP 5.3.2
- Baixe do site do Composer
- Funciona em qualquer plataforma
- Mais informações:

<https://getcomposer.org/doc/00-intro.md>

```
curl -s https://getcomposer.org/installer | php
```

Ou...

```
sudo apt install composer
```

Visão do consumidor: Atualização do composer

```
composer self-update
```

<https://getcomposer.org/doc/03-cli.md#self-update-selfupdate->

O que é um phar?

Simples:

phar = PHP Archive

<https://php.net/phar>

O que é um JSON?

JSON

JSON, um acrônimo para "JavaScript Object Notation", é um formato leve para intercâmbio de dados computacionais. JSON é um subconjunto da notação de objeto de JavaScript, mas seu uso não requer JavaScript exclusivamente. O formato JSON foi originalmente criado por Douglas Crockford e é descrito no RFC 4627. O media-type oficial do JSON é `application/json` e a extensão é `.json`.

[Mais em Wikipedia \(PT\)](#)

O que é um JSON?

Resumindo: uma linguagem para **formatar dados** de forma leve e legível para humanos.

Simples e de fácil aprendizado. Permite estruturar e formatar dados em **objetos**.

Instalando a primeira dependência

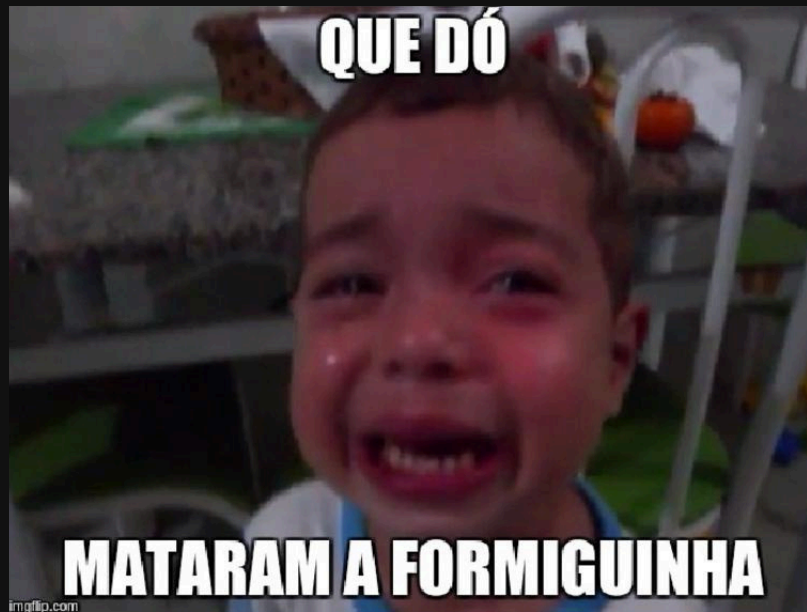
Crie um arquivo composer.json e coloque a sua dependência nele:

```
1 {  
2     "require-dev": {  
3         "vlucas/phpdotenv": "^4.0"  
4     }  
5 }
```

Instalando a primeira dependência

Crie um arquivo composer.json e coloque a sua dependência nele:

```
1 {  
2     "require-dev": {  
3         "vlucas/phpdotenv": "^4.0"  
4     }  
5 }
```



Instalando a primeira dependência

```
composer require <maintainer>/<dependency-name>
```

Exemplo:

```
composer require vlucas/phpdotenv
```

O que acontece após instalar?

Arquivos criados:

composer.json

Informações diversas sobre seu projeto,
dependências instaladas, etc

composer.lock

Todas as versões que foram instaladas

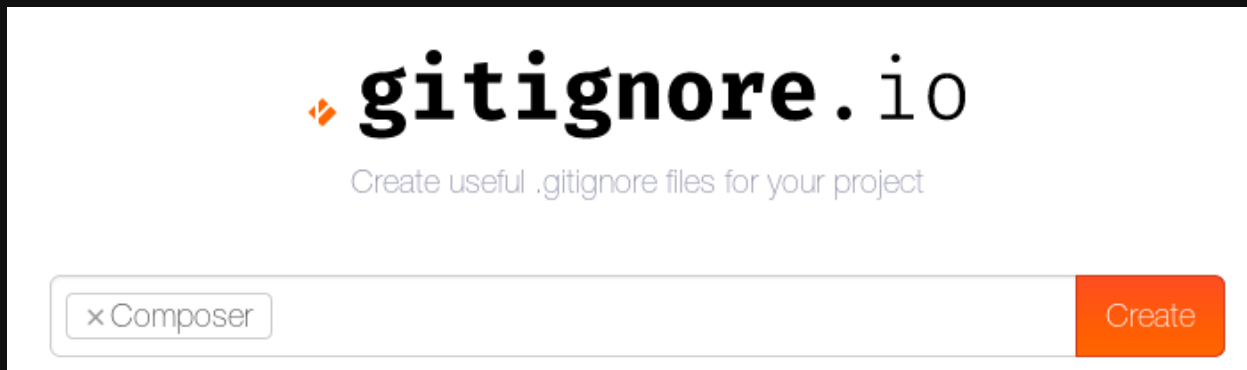
vendor

Todas as dependências instaladas

.gitignore

O que devemos ignorar?

<https://www.gitignore.io/>



composer.lock

Fixa as versões a serem instaladas garantindo que suas dependências e suas sub-dependências estão exatamente na mesma versão.

SEMPRE versione este arquivo!

Usando a primeira dependência

```
1 <?php
2
3 require_once 'vendor/autoload.php';
4
5 if (file_exists('.env')) {
6     $dotenv = Dotenv\Dotenv::createImmutable(__DIR__);
7     $dotenv->load();
8 }
9
10 echo getenv('TESTE');
```

require e require-dev

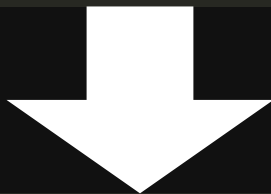
require é para ambiente de produção

```
1 {  
2     "require": {  
3         "php": ">=7.2.0",  
4         "cocur/sluggify": "^3.2"  
5     }  
6 }
```

require e require-dev

require-dev é APENAS para ambiente de produção

```
composer require phpunit/phpunit --dev
```



```
1 {  
2     "require-dev": {  
3         "phpunit/phpunit": "^8.4"  
4     }  
5 }
```

Exemplo: phpunit, behat, phpcs, phpcbf, phpmd, phpstan, etc

Versionamento semântico

1
major

2
minor

3
patch

Versionamento semântico

1
major

2
minor

Bugfixes

3
patch

Versionamento semântico

1
major

Novas
funcionalidades

2
minor

Bugfixes

3
patch

Versionamento semântico

Mudança de
arquitetura

1
major

Novas
funcionalidades

2
minor

Bugfixes

3
patch

Versionamento semântico - exemplos

- 1.0.0
- 1.0.2
- 1.1.0
- 0.2.5
- 1.0.0-dev
- 1.0.0-alpha3
- 1.0.0-beta2
- 1.0.0-RC5
- v2.0.4-p1

Instalar ou atualizar?

Instalar sempre, atualizar apenas quando necessitar
atualizar a versão das dependências

```
composer install
```

Estrutura do composer.json

Mantenedor

```
{
  "name": "vitormattos/blueprint-sdk-maker",
  "description": "Create SDK client from API Blueprint",
  "license": "MIT",
  "type": "library",
  "homepage": "https://github.com/vitormattos/blueprint-sdk-maker",
  "authors": [
    {
      "name": "Vitor Mattos",
      "homepage": "https://github.com/vitormattos"
    }
  ],
  "support": {
    "source": "https://github.com/vitormattos/blueprint-sdk-maker/tree/master",
    "issues": "https://github.com/vitormattos/blueprint-sdk-maker/issues"
  },
  "keywords": [
    "api",
    "blueprint",
    "parser",
    "apib"
  ]
}
```

Nome do pacote

Descrição

Licença

Tipo

Autores

Suporte

Palavras chaves

Estrutura do composer.json

Mantenedor

```
"autoload": {  
  "psr-4": {  
    "BlueprintSdkMaker\\": "src/Sdk"  
  }  
},  
"require": {  
  "php": ">=7",  
  "symfony/console": "^3.2",  
  "brianseitel/oasis-mson-parser": "dev-newest"  
},  
"require-dev": {  
  "nikic/php-parser": "^4.2",  
  "ext-tokenizer": "*",  
  "squizlabs/php_codesniffer": "3.*",  
  "overtrue/phplint": "^1.1",  
  "php-coveralls/php-coveralls": "^2.1",  
  "phpunit/phpunit": "^8",  
  "phpstan/phpstan": "^0.11.16"  
}
```

Regras de autoload

Namespace

Require

Versão do PHP

Require dev

Extensões

Estrutura do composer.json

Mantenedor

```
"scripts": {  
    "phpcs": "phpcs --standard=PSR2 -n src",  
    "phpcbf": "phpcbf --standard=PSR2 -n src",  
    "unit": "phpunit --coverage-clover ./tests/log/clover.xml --colors=always",  
    "phplint": "phplint ./ --exclude=vendor",  
    "phpstan": "phpstan analyse src --level 7",  
    "test": [  
        "@phplint",  
        "@unit",  
        "@phpcs"  
    ]  
},  
"bin": [  
    "bin/blueprint-suk-maker"  
],  
"repositories": [  
    {  
        "type": "vcs",  
        "url": "https://github.com/vitormattos/oasis-mson-parser"  
    }  
]
```

Scripts

Agrupando scripts

Binários

Repositórios

Tipo de repositório

Scripts

- **pre-install-cmd**: occurs before the install command is executed with a lock file present.
- **post-install-cmd**: occurs after the install command has been executed with a lock file present.
- **pre-update-cmd**: occurs before the update command is executed, or before the install command is executed without a lock file present.
- **post-update-cmd**: occurs after the update command has been executed, or after the install command has been executed without a lock file present.
- **post-status-cmd**: occurs after the status command has been executed.
- **pre-archive-cmd**: occurs before the archive command is executed.
- **post-archive-cmd**: occurs after the archive command has been executed.
- **pre-autoload-dump**: occurs before the autoloader is dumped, either during install/update, or via the dump-autoload command.
- **post-autoload-dump**: occurs after the autoloader has been dumped, either during install/update, or via the dump-autoload command.
- **post-root-package-install**: occurs after the root package has been installed, during the create-project command.
- **post-create-project-cmd**: occurs after the create-project command has been executed.

Scripts: exemplo Laravel

```
"scripts": {
  "post-root-package-install": [
    "@php -r \"file_exists('.env') || copy('.env.example', '.env');\""
  ],
  "post-create-project-cmd": [
    "@php artisan key:generate"
  ],
  "post-autoload-dump": [
    "Illuminate\\Foundation\\ComposerScripts::postAutoloadDump",
    "@php artisan package:discover"
  ]
}
```

Publique no packagist

Packagist

Browse

Submit

Create account

Search packages...

vitormattos/blueprint-sdk-maker

⬇ `composer require vitormattos/blueprint-sdk-maker`

Create SDK client from API Blueprint

E se encontrar um bug?

Fork, corrija e use a correção

Exemplo:

```
"repositories" : [{  
    "type" : "vcs",  
    "url" : "https://github.com/vitormattos/oasis-mson-parser"  
}]
```

Indo para produção

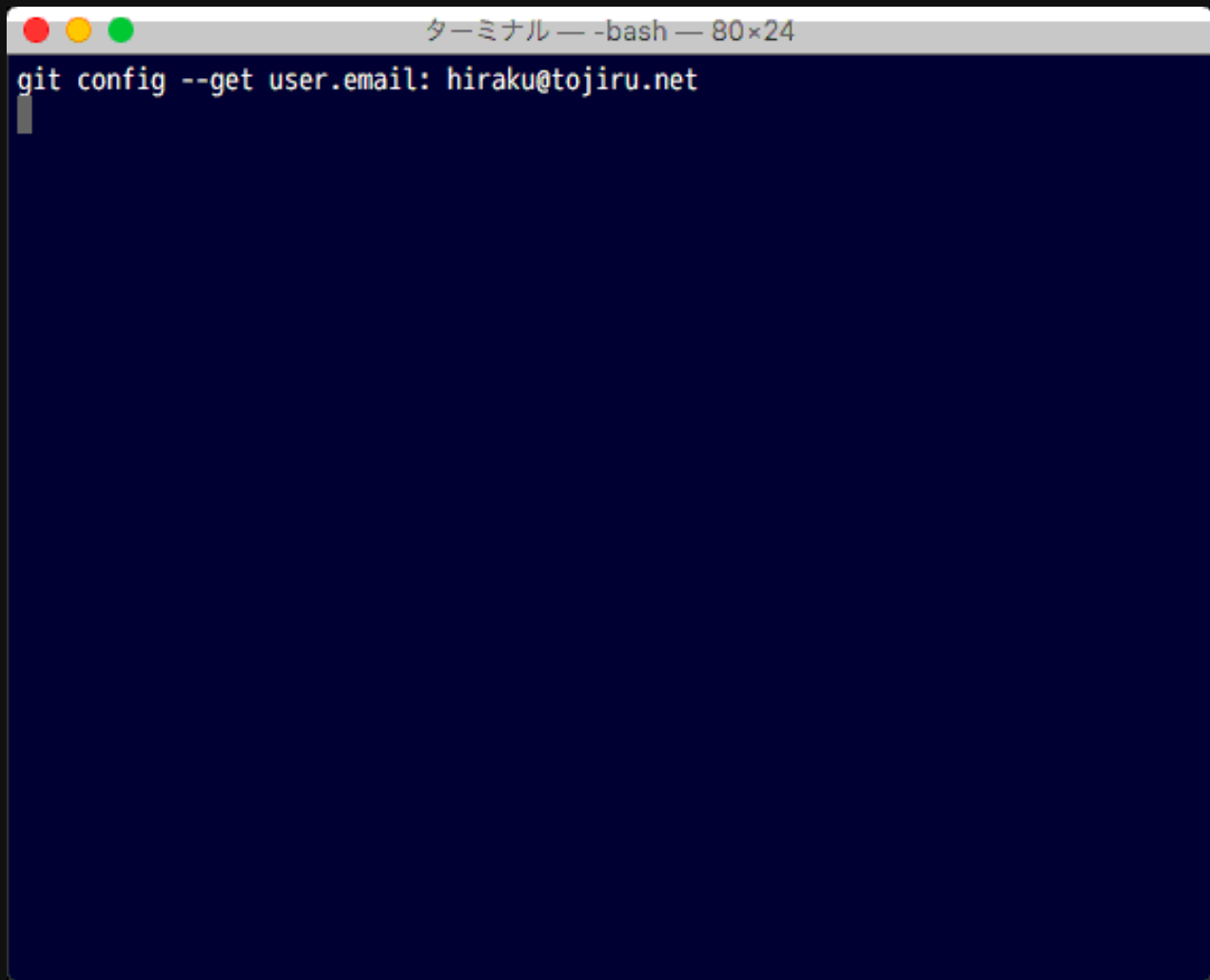
```
composer install --prefer-dist --no-dev --optimize-autoload
```

Plugins

hirak/prestissimo



Plugins

A terminal window with a dark blue background and a light gray title bar. The title bar contains three colored window control buttons (red, yellow, green) on the left and the text 'ターミナル — -bash — 80x24' on the right. The terminal displays the command 'git config --get user.email: hiraku@tojiru.net' in white text. A white cursor is positioned at the end of the command line.

```
ターミナル — -bash — 80x24  
git config --get user.email: hiraku@tojiru.net
```

Plugins

brainmaestro/composer-git-hooks

```
1 {
2     "extra": {
3         "hooks": {
4             "pre-commit": [
5                 "echo committing as $(git config user.name)",
6                 "php-cs-fixer fix ." // fix style
7             ],
8             // verify commit message. ex: ABC-123: Fix everything
9             "commit-msg": "grep -q '[A-Z]+-[0-9]+.*' $1",
10            "pre-push": [
11                "php-cs-fixer fix --dry-run ." // check style
12                "phpunit"
13            ],
14            "post-merge": "composer install"
15            "...": "..."
16        }
17    }
18 }
```

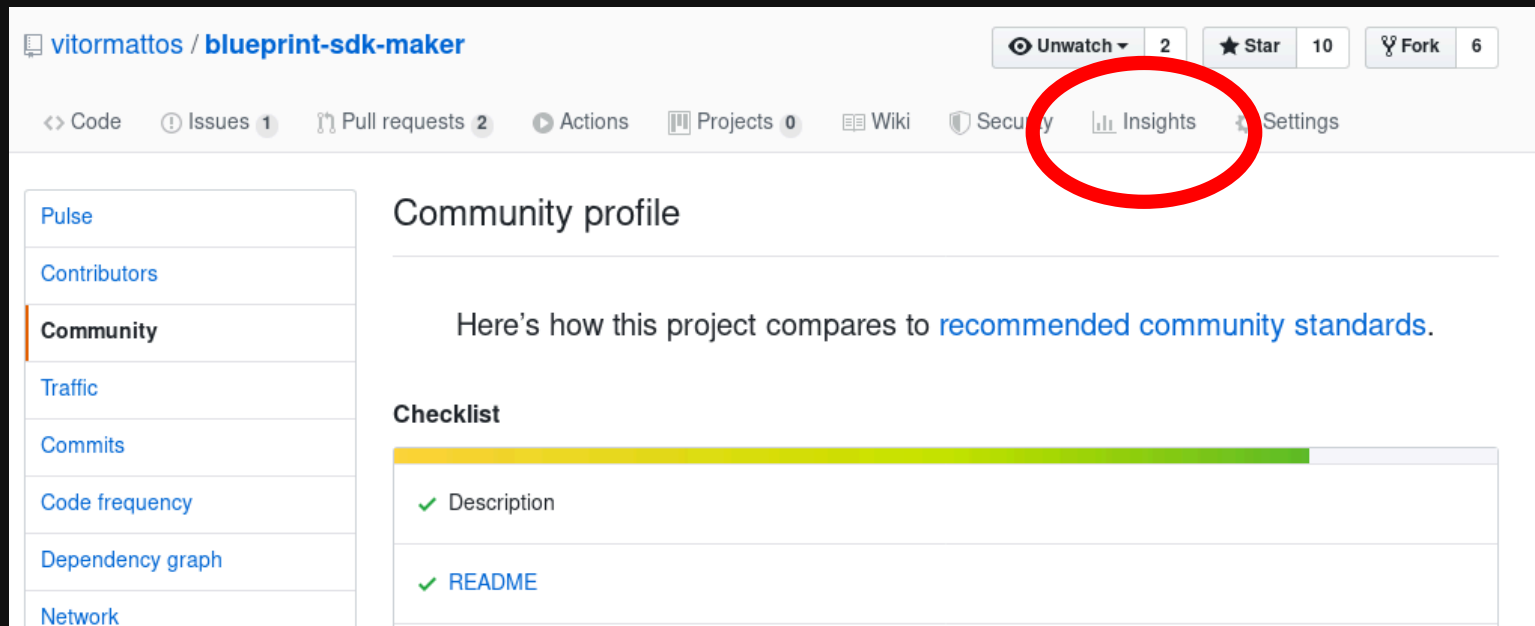
Um bom caminho: Boas práticas

<https://phppackagechecklist.com>

PHP Package Checklist

There's a lot that goes into a successful PHP package beyond simply having some useful code. The following checklist will help ensure that your package is taken seriously by the rest of the PHP community.

Um bom caminho: Boas práticas



Vitor Mattos

vitor@LT.coop.br
t.me/VitorMattos

